

17-12-24

AI lec no

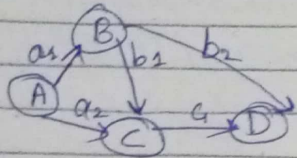
Tuesday

Markov Decision Process:

- Uncertainty Involved.

In search problems,

↳ given some state (S) & action (a)
we know deterministically, state $s'(s, a)$



Action is deterministic in search problems (not sure)

when uncertainty is there, search prob fails
here we talking about uncertainty (in real world) $\Rightarrow$ it involves randomness

state S, action a $\rightarrow$ random $\rightarrow$ state S'
probability involved.

Modeling in it is we used markov decision process.

Applications:

1) Robotics: actuators (left, right, up, down)
failure chances but probability is involved in robot actuators.

- decide where to move but actuators can fail.
- can hit a wall, etc

2) Resource Allocation:

Decide what to produce but:
Don't know customer demands for various products

$\rightarrow$ Agriculture: Decide what to plant but:
Don't know weather conditions.
Don't know crop yield

Example is:
 $\rightarrow$ Volcano Crossing Example:

	1	2	3	4	
1			-50	-50	
2			-50	-50	
3					

reward of 50 when we reach final

-50 $\Rightarrow$ stuck in lava (-50 reward)
2 $\Rightarrow$ local view, not scenic (2 reward)
20 $\Rightarrow$ target (scenic view)
start state (1, 1)
end state (3, 1), (1, 4), (1, 3), (2, 3)

Actions: $\begin{matrix} \uparrow N \\ \leftarrow S \end{matrix}$ Move N, move S, move E, move W

"if problem is search (no uncertainty) { simple exploration." $\rightarrow$ {E, S, S, E, E, N, N}

~~not a search problem~~ | if we have deterministic problem, we will have/get seq. of actions as a solution.

But:

In a reality, we are dealing with difficult / unseen terrain.

"can take wrong action".

"future reward ki ekhamrat is saai maloom hoti". future imp is less in hand is more if γ less than 1, future reward is less if γ more future reward value is more.

Differences:

Search Prob

1) $succ(s, a)$

↑

Deterministic

$succ(s, a)$ is a special case of $T(s, a, s')$

$$T(s, a, s') = \begin{cases} 1 & \text{if } succ(s, a) = s' \\ 0 & \text{other wise} \end{cases}$$

if probability is 1

MD-P

$T(s, a, s')$

↑
Probabilistic

2) $cost(s, a) \Rightarrow$ Reward (s, a, s')
obj: to minimize cost obj: to maximize reward

in Dice Game

$T(in, "stay", in) = 2/3$

$T(in, "stay", end) = 1/3$

$T(in, "Quit", end) = 1$

Transition:

s	action(a)	action(a)		$T(s, a, s')$
		Next state (s')	(Next state s')	
in	"stay"	in	in	2/3
in	"stay"	end	end	1/3
in	"Quit"	end	end	1

"Transition prob are distribution over states (s')"
successor nodes and sum of all prob on state (s) & sum of that nodes = 1

$$\sum_{s' \in \text{states}} P(s, a, s') = 1$$

$T(s, a, s') \geq 0$ ← always

→ Ex: Transportation example:

- streets with blocks 1 to n

- walking: s to $s+1$ takes 1 min

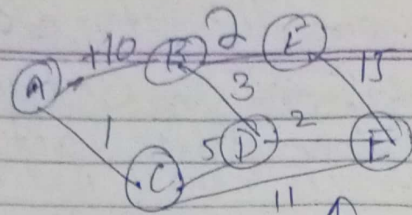
- tram: s to $2s$ takes 2 min

q) How to travel 1 to n in least amount of time? (above is search problem no probability).

→ tram can fail with prob = 0.5 = 50%
it becomes uncertain. → uncertainty.

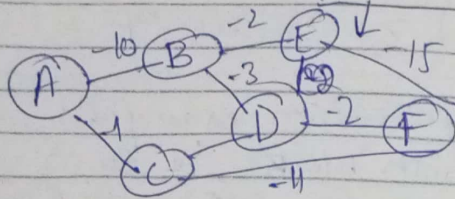
to convert min prob to max problem add (-) sign with reward sign.

Ex: Jx 1500,000
10,00,000 → minimize (search prob)
obj minimized
 Jx -1500,000
-10,00,000 → maximize (uncertainty)
obj maximized (markov)



min cost $A \rightarrow C \rightarrow D \rightarrow F = 8$

max cost $\leq A \rightarrow C \rightarrow D \rightarrow F = -8$



What is a solution in MDP?

What is sol in search $\rightarrow$ seq of actions

$\rightarrow$ In MDP: Policy (representation of policy)

A policy (π) is a mapping from each state (s) $\in$ states to an action (a) $\in$ Actions.

Ex: in volcano crossing (my policy can be):

State	Action
(1,1)	S $S \rightarrow$ South
(2,1)	E
(1,2)	S
(2,3)	N
...	...

to check it is a good policy or not (more reward required).

$\pi(s) \rightarrow$
 $\pi: S \rightarrow A(s)$
 to evaluate policy, we can do that:

if $\gamma = 0$, future ignore
 if $\gamma = 1$, total future value is }
 very imp.

19-12-24

AI lee

Thursday

from policy:

 $\pi(s)$ $\pi: S \rightarrow A(s)$

→ policy can be given or you can find policy.

• normally have to find policy

→ suppose we know the policy

& we want to see how good it is
 • ex. dice game, policy is stay, evaluate whether it is good or not

→ Policy Evaluation:

Evaluating the policy:

following a policy gives us random path. (bit of randomness in policy).

"for quit, there is no random path"

Def: utility - utility of a policy is discounted sum of rewards on that path. (this is a random quantity).

$$\text{utility } U = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

↑
immediate
reward

↑
future
reward

= if $\gamma = 1$ (future value is same to)

Example:
following a policy $\pi = \text{stay}$

path	Utility
[in: stay, 4, end]	4
[in: stay, 4, in: stay, 4, in: stay, 4, end]	12
[in: stay, 4, in: stay, 4, end]	8
⋮	⋮

we find expected utility

Def: value (Expected utility) $\leq E(U)$
 the value of policy is expected utility

$E(U) \rightarrow$ average of different utilities

Discounting γ
 $\rightarrow$ if $\gamma = 1 \rightarrow$ (saving for future)

$$E(U) = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

$$= 4 + 1 \cdot 12 + 1^2 \cdot 8 + \dots$$

$\rightarrow$ if $\gamma = 0$ (living in a moment)

$\hookrightarrow$ it can be changed bko $(0 \leq \gamma \leq 1)$

if policy (π) is given
 value of policy:

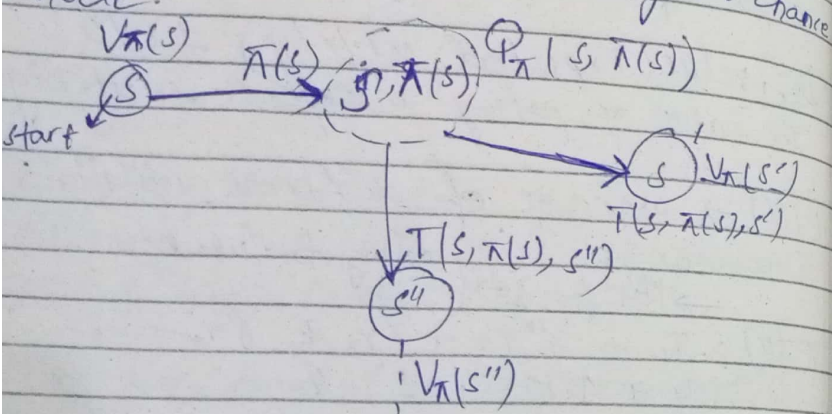
let, $V_\pi(s) \rightarrow$ be expected utility
 by following π (policy) from state (s).

We can find every state's value of policy

$V_{\pi}(s) \rightarrow$ value of policy at state s .
 Def: Q -value of policy: (chance node, prob)

let $Q_{\pi}(s, a)$ be expected utility of taking an action (a) and then following policy (π).
 "means in action $a \rightarrow$ stay & then follow policy"

$Q_{\pi}(s, a) \rightarrow$ Expected utility at chance node.



Def. Now we define Recurrence:

$$V_{\pi}(s) = \begin{cases} 0 = \text{no action, no reward} & \text{if is End}(s) \\ Q_{\pi}(s, A(s)) & \text{otherwise} \end{cases}$$

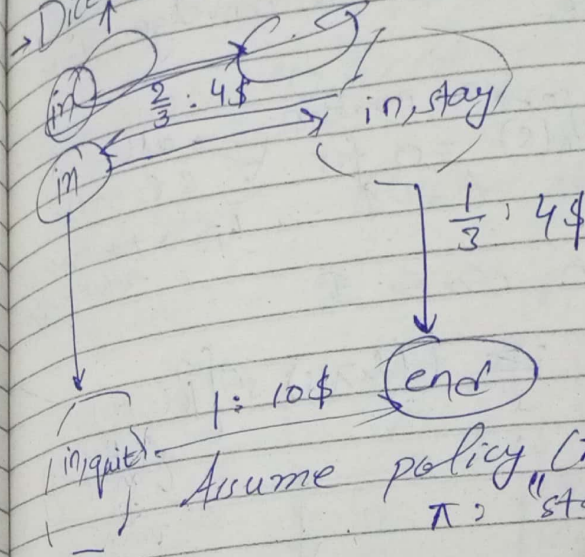
$$Q_{\pi}(s, a) = \sum_{s' \text{ state}} T(s, a, s') [\text{Reward}(s, a, s') + \gamma V_{\pi}(s')]$$

$\downarrow$ immediate reward
 $\downarrow$ discounted reward

$$Q_{\pi}(s, a) = \sum_{s' \text{ state}} T(s, a, s') [\text{Reward}(s, a, s') + \gamma V_{\pi}(s')]$$

$\downarrow$ next state

Game Example for Policy Evaluation:



Assume policy (π) is given:
 π : "stay"

$\pi(\text{in}) = \text{"stay"}$
 $V_{\pi}(\text{in}) = Q_{\pi}(\text{in}, \text{stay})$ [from previous page]
 $V_{\pi}(\text{end}) = 0$

$$V_{\pi}(\text{in}) = \frac{1}{3} (4 + V_{\pi}(\text{end})) + \frac{2}{3} (4 + V_{\pi}(\text{in}))$$

$$= \frac{1}{3} (4) + \frac{2}{3} (4 + V_{\pi}(\text{in}))$$

$V_{\pi}(\text{in}) = 12 \$$ using closed form sol (by solving eq).

in reality we have no of states & closed form sol is harder
 so we use another method:
 → Iterative Algorithm for Policy Evaluation:
 - start with arbitrary policy & repeatedly apply recurrence to converge to true value.

Alg:

Initialize $V_{\pi}(s) = 0$ for $\forall s \in \text{states}$
 for iteration $t = 1$ one iteration
 for each state s all $s \in \text{states}$
 $V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s')]$
total policy evaluation step

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s')]$$

previous step value

$$V_{\pi}(s, \pi(s))$$

Time Complexity (3 loops): $O(t \cdot p \cdot |S| \cdot |S'|)$
no of states, no of states

How many iterations (value of t):
 $t_{pe} = ?$
 Repeat your iterations until values don't change much

$$|V_{\pi}(s) - V_{\pi}^{(t-1)}(s)| \leq \epsilon \rightarrow \text{absol error}$$

$\epsilon = 10^{-5}$ Assumed value

$$\max_{s \in \text{states}} |V(s) - V_{\pi}(s)| \leq \epsilon$$

jab so barhio value hai wo less than / equal ho ϵ "kai"

Using Iterative Alg for Policy Evaluation in Dice Game:

Let $\pi(\text{in}) = \text{'stay'}$
 $V_{\pi}^{(t)}(\text{end}) = 0$ $\gamma = 1$

t	$V_{\pi}^{(t)}$	end	in
0	$V_{\pi}^{(0)}$	0	0
1	$V_{\pi}^{(1)}$	0	$4 \times \frac{1}{3} + \frac{2}{3} V_{\pi}^{(0)}(\text{in}) = \frac{1}{3} [4 + 0] = \frac{4}{3}$
2	$V_{\pi}^{(2)}$	0	$132/3 = 44$
3	$V_{\pi}^{(3)}$	0	$\frac{1}{3} [4 + \frac{4}{3}] = \frac{16}{9}$

$$V_{\pi}^{(1)}(\text{in}) = \frac{1}{3} [4 + V_{\pi}^{(0)}(\text{end})] + \frac{2}{3} [4 + V_{\pi}^{(0)}(\text{in})]$$

$$= \frac{4}{3} + \frac{2}{3} [4 + 0] = \frac{4}{3} + \frac{8}{3} = \frac{12}{3} = 4$$

$$V_{\pi}^{(2)}(\text{in}) = \frac{1}{3} [4 + V_{\pi}^{(1)}(\text{end})] + \frac{2}{3} [4 + V_{\pi}^{(1)}(\text{in})]$$

$$= \frac{4}{3} + \frac{2}{3} [4 + \frac{4}{3}] = \frac{4}{3} + \frac{10}{3} = \frac{14}{3}$$

$$V_{\pi}^{(3)}(\text{in}) = \frac{1}{3} [4 + V_{\pi}^{(2)}(\text{end})] + \frac{2}{3} [4 + V_{\pi}^{(2)}(\text{in})]$$

$$= \frac{4}{3} + \frac{2}{3} [4 + \frac{14}{3}] = \frac{4}{3} + \frac{20}{3} = \frac{24}{3} = 8$$

24-12-2024

AI lec

Tuesday

utility = when running one time
 expected utility = when running many times, its reward.

For

↳ given policy $\pi \rightarrow$ we can evaluate/use policy evaluation alg to evaluate the policy.

Now:

How to find the policy π ?

↳ to get maximum reward

↳ target: find a policy to get maximum expected utility $E(U)$

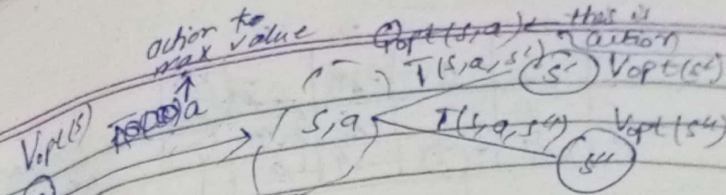
$\rightarrow$ to get optimal value & its policy.

$\rightarrow$ try to get directly $\uparrow$ $E(U)$

optimal value means, it is a max value obtained by any policy.

optimal value & optimal Q value:
 previously $\rightarrow V_{\pi}(s), Q_{\pi}(s,a)$

$V_{opt}(s), Q_{opt}(s,a)$
 optimal



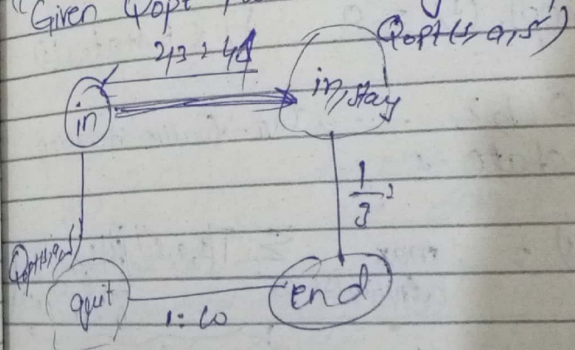
find $V_{opt}(s) = ?$

optimal value at state s from state s
 $V_{opt}(s) = \begin{cases} 0 & \text{if end}(s) = 1 \\ \max_{a \in \text{Action}(s)} Q_{opt}(s,a) & \text{otherwise} \end{cases}$

chance node (after that we have chance node)

$$Q_{opt}(s,a) = \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma V_{opt}(s')]$$

"Given Q_{opt} , we can get optimal policy."



$$(1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow \dots$$

$$\pi_{\text{opt}}(s) = \arg \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$$

→ Algorithm: Value Iteration.
[Bellman in 1957] → proposed by
it is used to find optimal policy

initialize $V_{opt}^{(0)}(s) = 0 \quad \forall s \in \text{States}(S)$

for $t = 0, 1, 2, \dots, t_{\text{max}} \rightarrow (\text{value iteration})$
for each state s :

$$V_{opt}^{(t)}(s) = \max_{a \in \text{Action}(s)} \sum_{s'} T(s, a, s') \left(R(s, a, s') + \gamma V_{opt}^{(t-1)}(s') \right)$$

time complexity $O(t_{\text{opt}} \cdot |S| \cdot |A| \cdot |S|)$

→ Taking max over actions we can find
Opt (s)

don't know rewards, but
do that actions

→ In real life, we don't know rewards, but with passage of time, will do that actions from which reward is maximized. we also have uncertainty.

→ episode? $\text{policy}(\pi) = \text{stay}$
 episode $\leftarrow \{in; \text{stay}, 4, \text{end}\}$
 episode $\leftarrow \{in; \text{stay}, 4, in; \text{stay}, 4, in; \text{stay}, 4, \text{end}\}$
 so, a policy, will have
 episode.

episode $\leftarrow$ path
when executes a policy, will have
diff policy $\uparrow$ which is episode.

$$V_A(s) = P$$

$$V_{opt}(s) = P$$

MDP:

states
→ start states

→ Action (✓)

$$T(s, a, s')$$

$$R(s, a, s')$$

→ is end (S)

→ discounted factor (δ)

in Real world

Unknown transition prob & rewards

if you don't know trans prob, R_{150}
you do brainstorming if you know the
problem. so it is

MDP (offline learning)	(online learning) RL
- we have mental model of how real world works	- we don't know how world works.
- find the policy that maximizes the rewards. (don't learn transitional probs & rewards).	- perform actions in the world to find out & collect rewards.

The diagram illustrates the interaction between an Agent and an Environment. The Agent is on the left, and the Environment is on the right. A horizontal arrow points from the Agent to the Environment, labeled "actions (a)". A horizontal arrow points from the Environment back to the Agent, labeled "reward (r)" and "new state (s')".

→ Algorithm: policy $\pi_t \Rightarrow$ actual max ^{value} π_t
for $t = 1, 2, 3 \dots$
choose an action $(a)_t = \pi_{act}(s_{t-1})$ (flow)

update parameters (model)
transitional prob & Rewards.

Monti Carlo Method:
→ based on Monte

→ Model based MDP: $(T(s, a, s'), R(s, a, s'))$
 of estimator the MDP: $(T(s, a, s'), R(s, a, s'))$
 (Based on data).
 $s_0, a_1, r_1, s_1; a_2, r_2, s_2; a_3, r_3, s_3$
 we have to find

Data: $\{data\}$ and from this can get T & R .
from r_1, r_2 we ~~have~~ R but transit we have to estimate.

Transitions: $\uparrow \notin (s, a, s') = \#time(s, a, s')$
occurs
 $\#time(i, a) \text{ occurs}$

Example from Dice Game

We have to generate data and for that we need some policy.

if we know on what policy we want to generate data, it is on policy learning.

Policy: Stay $\rightarrow$ data generating process

Data 1:

[in; stay, 4; in; stay, 4; in; stay, 4; in; stay, 4; end]

$$T(\text{in}, \text{stay}, \text{in}) = \frac{3}{4} \rightarrow \text{total ways in}$$

$$T(\text{in}, \text{stay}, \text{end}) = \frac{1}{4} \rightarrow \text{total stay}$$

$$R(\text{in}, \text{stay}, \text{in}) = 4 \text{ \pounds}$$

$$R(\text{in}, \text{stay}, \text{end}) = 4 \text{ \pounds}$$

Sample data 2:

[in; stay, 4; in; stay, 4; end]

add it in pure water

$$T(\text{in}, \text{stay}, \text{in}) = \frac{4}{6}$$

$$T(\text{in}, \text{stay}, \text{end}) = \frac{2}{6}$$

sample 3:

sample 4:

...

Problem of this is:

If we choose one policy, we cannot find parameters for another policy (i.e., quit policy).
 $\rightarrow$ for that we do exploration.

Exploration

Problem: won't see all (s,a) pairs
 $\rightarrow$ do exploration/choose all policies.

page table

2-1-2025

AI lec

Thursday

(2) Approach No # 02

actual target is optimal Policy
can be get by Q_p for that
we use model-free learning.

→ Model-Free Monte Carlo Methods:

$\hat{Q}(s, a) \leftarrow \text{find it} = \sum_{s' \in \text{States}} f(s, a, s') [R + \gamma V_{\text{opt}}(s')]$
estimate it

↑
[?] don't know

we will directly find $\hat{Q}_{\text{opt}}$ instead of $f \& \hat{V}$

Try to estimate $Q_{\text{opt}}(s, a)$ directly.

→ Data following a policy π → gives us data

$s_0; a_1, r_1, s_1; a_2, r_2, s_2; a_3, r_3, s_3; \dots$

$Q_{\text{opt}}(s, a) \Rightarrow$ optimal value of a policy after taking an action a .

→ utility: U

- accumulated policy.
- random variable.

$$U_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$\downarrow$ immediate reward $\downarrow$ discounted sum reward

estimate

$Q_{\text{opt}}(s, a) = \text{average of } U_t$
where $t-2 = s, U_t = a$

ex:
Policy $\pi(s)$

$\dots s, a, \dots$

$Q_{\text{opt}}(s, a) = ?$

just hi (s,a) aya w sa agai U_t ayegi

if (s,a) occurs before, after, w sa agai U_t calculate hoga.

episode $s, a \xrightarrow{\quad} U_t$

$\hookrightarrow$ then take avg of all U_t values

$Q_{\text{opt}}(s, a) = \arg U_t$

Policy $\pi = \text{stay}$ (Assume $\gamma = 1$)

$[\text{in; stay, 4; end}] \rightarrow U_t = 4$

$[\text{in; stay, 4; in; stay, 4; in; stay, 4; end}]$

$[\text{in; stay, 4; in; stay, 4; end}]$

$$U_t = 4 + 4 + 4 = 12$$

$$U_t = 4 + 4 = 8$$

$$Q_{\pi}(\text{in, stay}) = \frac{4 + 12 + 8}{3} = 8$$

all states are not explored yet.
Algorithm:

→ on-policy Alg →
→ off-policy Alg

on-policy Alg: Free Model Based Monte Carlo (based on data). data is generated on basis of specific policy.

off policy: Model Based Monte Carlo (not specific data / policy).

Model Free M.C:

$$\hat{Q}_n(s,a) = \text{avg } U_t$$

estimate values

$$\hat{Q}_n(s,a) = \underset{\substack{\downarrow \\ \text{old} \\ \text{value}}}{\hat{Q}_n(s,a)} + \underset{\substack{\downarrow \\ \text{new} \\ \text{value}}}{U}$$

Run-time avg is calculated.

$$\hat{Q}_n(s,a) = (1-\gamma) \hat{Q}_n(s,a) + \gamma U$$

$$\gamma = \frac{1}{1 + \text{no of updates to } (s,a)}$$

$$\gamma = \frac{1}{1 + \text{no of updates}}$$

ignoring (s,a)

Ex:

Data: $U = 4, 1, 1, \dots$

$$\hat{Q}_n(s,a) = (1-\gamma) \hat{Q}_n(s,a) + \gamma U$$

initially # of upd = 0

$$U_1 = (1-1) \hat{Q}_n + 1 \cdot (U) = 4$$

$$U_2 = \hat{Q}_n(s,a) = \underset{\substack{\downarrow \\ \text{old} \\ \text{value}}}{\frac{1}{2}(4)} + \underset{\substack{\downarrow \\ \text{new} \\ \text{value}}}{\frac{1}{2}(1)} = \frac{5}{2}$$

$$U_3 = \hat{Q}_n(s,a) = \frac{2}{3} \left(\frac{5}{2} \right) + \frac{1}{3}(1) = \frac{6}{3}$$

$\therefore \gamma = \frac{1}{3}$

→ Equivalence notation (G.D)

$$\hat{Q}_n(s,a) = \hat{Q}_n(s,a) - \gamma [\underset{\substack{\downarrow \\ \text{previous}}}{\hat{Q}_n(s,a)} - \underset{\substack{\downarrow \\ \text{target}}}{U}]$$

↕ equivalent

$$\hat{Q}_n(s,a) = (1-\gamma) \hat{Q}_n(s,a) + \gamma U$$

→ (equal to gradient Descent)

will have 4 values, which is max, go to that side

X	X	-50	50
X	X	-50	X
2	X	X	X

$$\left. \begin{aligned} Q_n((1,1), N) &= 3 \\ Q_n((1,1), S) &= \\ w &= \\ \epsilon &= \end{aligned} \right\} \text{which is maximum, go that way.}$$

$\bar{A}$	Action
(1,1)	N
(1,2)	—
(2,1)	—
..	..
..	..

if policy & prob given, execute it by executing it, we get U_t

→ Q-learning Alg.
on each (S, a, r, s') → (will have 4 possible values for volcano)

find Q_{opt} from above.

$$Q_{opt}(s, a) = (1 - \gamma) Q_{opt}(s, a) + \gamma (r + Q_{opt}(s'))$$

$$V_{opt}(s) = ?$$

where

$$V_{opt}(s) = \max_{a \in \text{Action}(s)} Q_{opt}(s, a)$$

1	2
3	4

optimal value → gives max value at s
if this gives max, then that value is $V_{opt}(s)$

for first iteration it will be 0.

→ Reinforcement learning:

— covering the unknowns.

for iterations $t = 1, 2, 3, \dots$

choose action $A_t = \bar{A}_{act}(S_{t-1})$ → How?
Receive reward r_t & observe s_t
update parameters (how)

Q which policy $\bar{A}_{act}$ to use?

Attempt - 1: Set $\bar{A}_{act}(s) = \arg \max$

$$\arg \max_{a \in \text{Action}(s)} Q_{opt}(s, a)$$

→ No exploration, only exploitation by this policy (John Sahi waha gaye greedy approach).

• can stuck on local minima.

(considering $\hat{Q}_{opt}$)

→ Attempt - 2: No exploitation, all exploration.

π is a random policy.

$\pi_{act}(s) = \text{random}$ (not considering $\hat{Q}_{opt}$)

Random ~~functions~~ Actions (s)
not converging for bestest value.

So, there should be trade off b/w exploitations and explorations.

Epsilon greedy strategy / policy is used here.

$\pi_{act}(s) = \begin{cases} \arg \max_{a \in \text{Actions}(s)} \hat{Q}_{opt}(s, a) & \text{prob} = 1 - \epsilon \\ \text{rand for Action}(s) & \text{with prob } \epsilon \end{cases}$

if $\epsilon = 0$, exploitation
if $\epsilon = 1$, exploration
if $\epsilon = 0.5$, balance
exploration more in shury & exploitation more gradually.

1-2025

AI/ML

Tuesday

→ Games:

→ you have an opponent in this game

→ Adversarial Games:

A simple game:

we have 3 bins

A
-5, 50

B
1, 3

C
-5, 15

→ you choose one of the bins.

→ I choose number from that bin.

- your goal is to maximize the chosen number

if other is adversary you will choose B b/c it will get less that is 1, can't get negative no, but if other is cooperating, you choose A, but if you don't know him (beh is random) so you choose avg value which will be C.

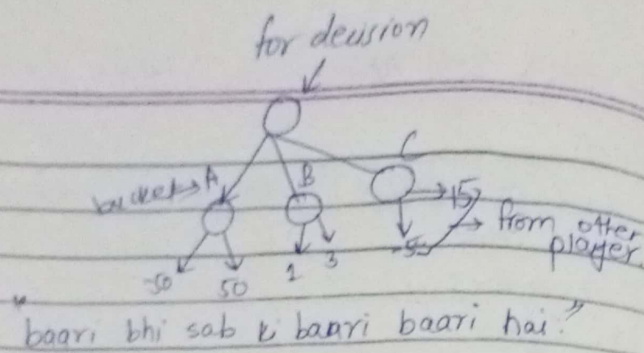
Adversary → he will take worst step

• they are similar to state based Model.

diff is we have game tree.

- in this, each node is a decision point.

- each root to the leaf node path is a possible outcome of the game.



- From root to leaf, we get outcome.
- Games, we have players (2 or more)
- state of game is fully observed (knows the state of game).
- Assume $\rightarrow$ Two players (Agent, Opponent)
- zero sum game & when goes the utility of agent is negative of utility of opponent.
"agar ap jeeta to 1000 and opponent haara to -1000 wesy"

$\rightarrow$ Two player zero sum game:

- formal Definition:
- s_{start} - start state
- $Actions(s)$ - possible actions from state(s)
- Successor - $su(s, a)$ -
- $isEnd(s)$ -
- Utility (s) - Agents utility for end states.
- player (s) $\in$ Players - Player who control state (s).

- Each state has designated player
- $\rightarrow$ Diff b/w state based model &

- Adversarial Games:
- Each state has designated players
- instead of last of edges, we have utility defined at End state.

$\rightarrow$ Example:

- chess game:
- players $\in$ {white, black}
- state(s) = position of all pieces, whose turn it is.
- Actions(s): legal chess moves at state(s).
- $isEnd(s)$: whether s is checkmate / draw.
- utility(s): $+\infty$ if white player wins &
 $-\infty$ if black player wins.

$\rightarrow$ Policies: (it is actually state-action mapping)

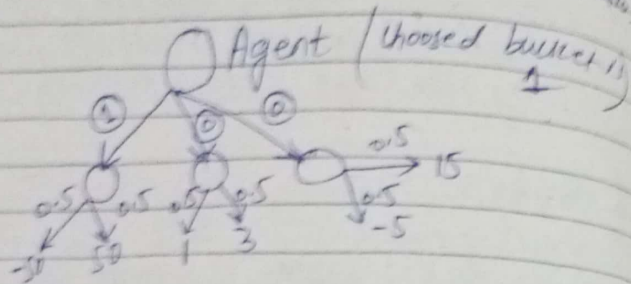
- policy $\pi_p(s) \in Actions(s)$
(he will tell which action he will perform).

Actions that player p takes in state (s).

- Stochastic policy $\pi_p(s, a) \in [0, 1]$
uncertainty (will return probability)
probability of a player p taking an Action (a) at state (s)

- Game Evaluation Example:

Agent (1) $\in A$ A bin (previous example)
 $T_{\text{opp}}(s) = \frac{1}{2}$ for $a \in \text{Actions}(s)$
 only has two choices b/c of two



"choosing A with 100% prob"
 "opponent bet is stochastic."

$V_{\text{eval}}(s_{\text{start}}) = ?$

Evaluation is bottom-up.

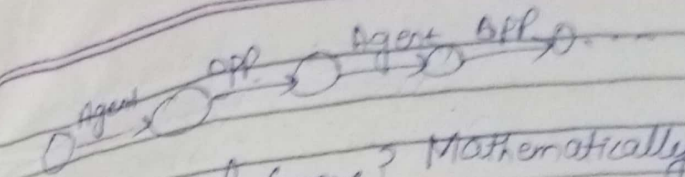
$$= \frac{1}{2} [0.5(-50) + 0.5(50)] + \frac{1}{2} [0.5(1) + 0.5(3)] + \frac{1}{2} [0.5(-5) + 0.5(15)]$$

$$= 0 + 0 + 0 = 0$$

(we can evaluate value of game by GameTree)

→ Define it Mathematically:

→ Game Evaluation Recurrence:



find value of Game? Mathematically
 $V_{\text{eval}}(s) = ?$

if we are at $\text{isEnd}(s)$, we get utility

$$V_{\text{eval}}(s) = \begin{cases} \text{Utility}(s) & \text{isEnd}(s) \\ \sum_{a \in \text{Action}(s)} T_{\text{agent}}(s, a) * V_{\text{eval}}(\text{succ}(s, a)) & \text{player}(s) = \text{Agent} \\ \sum_{a \in \text{Action}(s)} T_{\text{opp}}(s, a) * V_{\text{eval}}(\text{succ}(s, a)) & \text{player}(s) = \text{opp} \end{cases}$$

→ opponent doing

$$\frac{1}{2} [0.5(-50) + 0.5(50)]$$

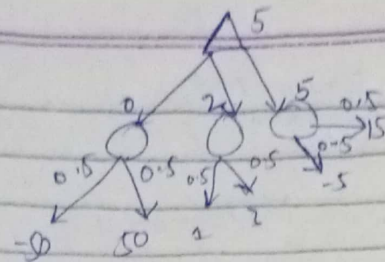
→ agent doing

→ Expectimax Example:

Utility of agent is doing too & opponent trying to get utility of -∞
 (i) opponent is stochastic (random) → so that is Expecti.

$$T_{\text{opp}}(s, a) = \frac{1}{2} \text{ for } a \in \text{Actions}(s)$$

"agent policy is to maximize utility."



$\Delta \rightarrow \text{max} \rightarrow \text{Agent}$
 $\bigtriangledown \rightarrow \text{min} \rightarrow \text{opponent}$
 $\bigcirc \rightarrow \text{stochastic}$

$$V_{\text{eval}}(s) = ?$$

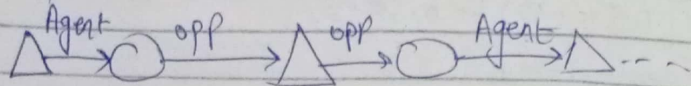
We have to maximize, so we go to C bin

$$V_{\text{eval}}(s) = 0.5(-5) + 0.5(15) = 5 \text{ so, it will choose Bin (C)}$$

"you are maximizing & opponent is stochastic"

"on avg we will choose 5 / Bin C"

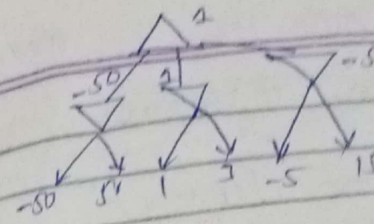
→ Expectimax Recurrence:



$$V_{\text{expectimax}}(s) = \begin{cases} \text{utility}(s) & \text{if isend}(s) = T \\ \max_{a \in \text{Action}(s)} V_{\text{expectimax}}(\text{succ}(s, a)) & \text{if player}(s) = \text{Agent} \\ \sum_{a \in \text{Action}(s)} \lambda(a) * V_{\text{expectimax}}(\text{succ}(s, a)) & \text{if player}(s) = \text{opponent} \end{cases}$$

→ Minimax Example 2

agent is maximizing, opp min

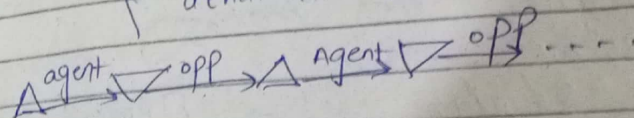


$$V_{\text{eval}}(s) = 1 \text{ (so go to Bin B)}$$

b/c opp is min

→ Recurrence of Minimax:

$$V_{\text{minimax}}(s) = \begin{cases} \text{utility}(s) & \text{if is End}(s) = T \\ \max_{a \in \text{Action}(s)} V_{\text{minimax}}(\text{succ}(s, a)) & \text{if } P(s) = \text{Agent} \\ \min_{a \in \text{Action}(s)} V_{\text{minimax}}(\text{succ}(s, a)) & \text{if } P(s) = \text{opp} \end{cases}$$



14-1-2025

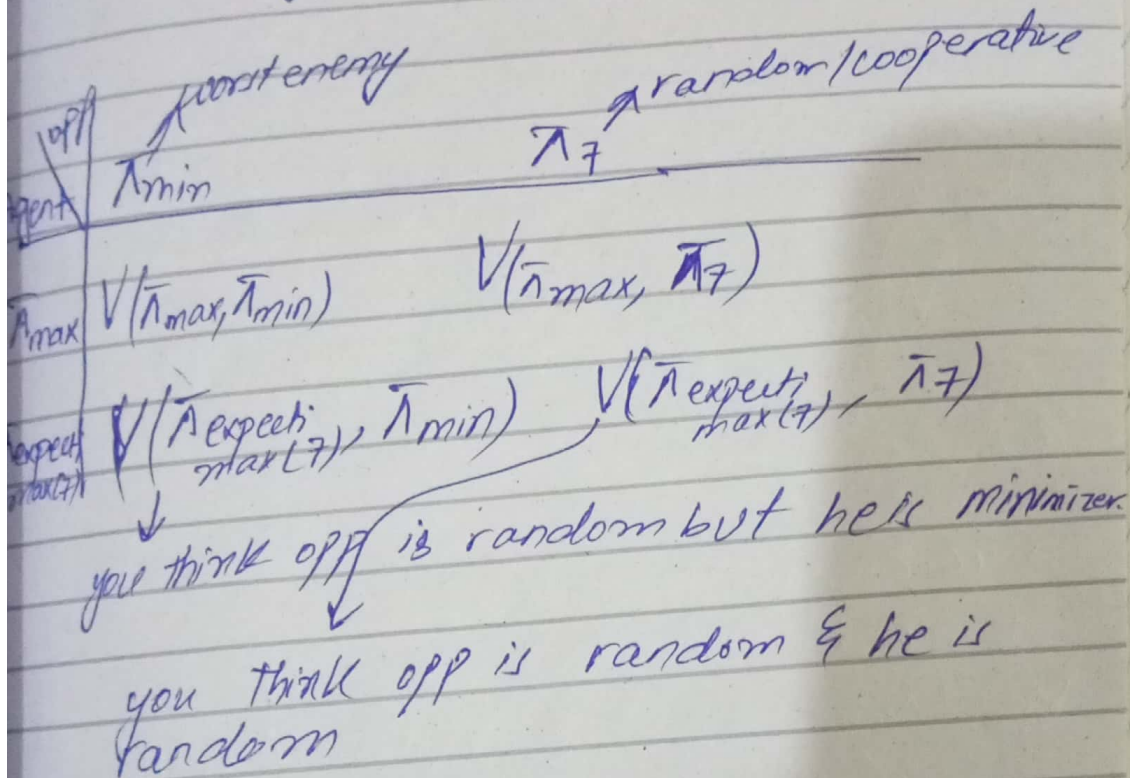
AI lec

Tuesday

Properties of Minmax:

Vexpectimax - Agent is following max policy
opp has random behaviour. π_7

Vminimax - Agent (π_{max}), opp - π_{min}



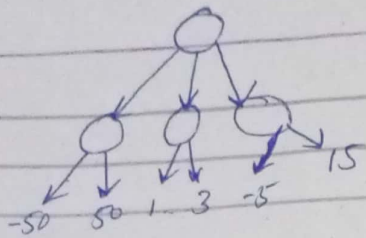
→ Minimax property - 1:

- Best against minimax opponent

$$V(\pi_{max}, \pi_{min}) \geq V(\pi_{agent}, \pi_{min}) \text{ for all agents}$$

you will get max value than other this value. (agent is not max here)

Bin example:



$$V(\pi_{\max}, \pi_{\min}) = 1$$

$$V(\pi_{\text{expecti}_{\max(\pi)}, \pi_{\min}}) = -5$$

he will get random so give avg of all values, will try to go to bin he will give (-5) b/c opp is min.

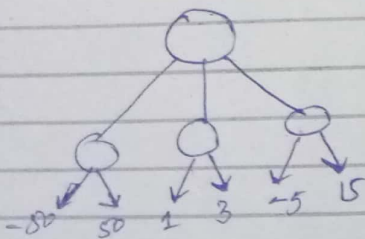
So $V(\pi_{\max}, \pi_{\min}) > V(\pi_{\text{expecti}_{\max(\pi)}, \pi_{\min}})$
So if opp is min then strategy (i) is best.

→ minimax property-2:

- lower bound against any opp

$$V_{\max}(\pi_{\max}, \pi_{\min}) \leq V(\pi_{\max}, \pi_{\text{opp}}) \text{ for all } \pi_{\text{opp}}$$

ya min he ya behtar (random)



$$V(\pi_{\max}, \pi_{\min}) = 1$$

opp is changing the policy from $\pi_{\min}$ to π_{π} keeping in mind that opp is min.

$$V(\pi_{\max}, \pi_{\pi}) = 2 \rightarrow \text{from bin 2 avg.}$$

→ minimax property : 3:
- Not optimal if opponent is known
- hamon pata zero or wo random but we consider it as min.

$$V(\pi_{\max}, \pi_{\pi}) \leq V(\pi_{\text{expecti}_{\max(\pi)}, \pi_{\pi}}) \text{ for opponent } \pi_{\pi}$$

$$V(\pi_{\max}, \pi_{\pi}) = 2 \rightarrow \text{from bin 2 avg}$$

$$V(\pi_{\text{expecti}_{\max(\pi)}, \pi_{\pi}}) = 5 \rightarrow \text{from bin 3 avg}$$

if you know opp but you set it as worst you will get less value where as if you know policy and you play it you get more value than above policy.

→ Modified Game:

- you choose one bin out of three.
- flip a coin, if head, then move one bin to the left (with wrap around) → circular.
- I chose a no from that bin
- your goal is to maximize the chosen no.

(A)

-50, 50

(B)

1, 3

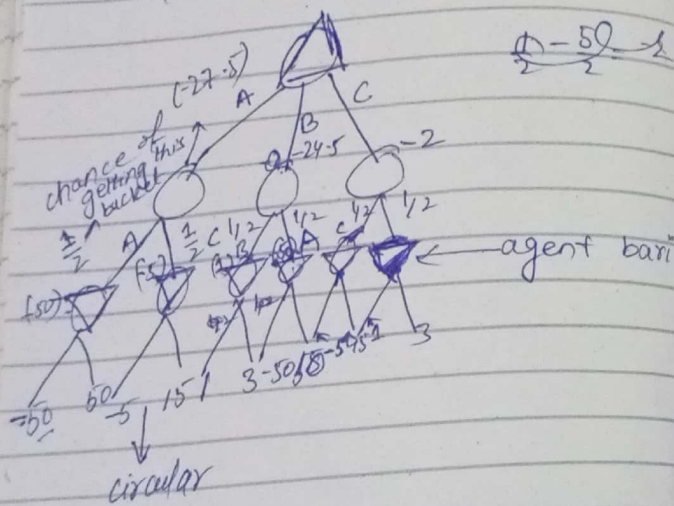
(C)

-5, 15

so we have 3 player,
agent, opl, nature (head, tail).

Example:-
Expectiminimax:

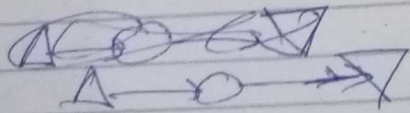
$$\pi_{\text{coin}}(s, a) = \frac{1}{2} \text{ for } a \in \{0, 1\}$$



$$V_{\text{Expectimax}}^{\text{mini}}(\text{start}) = ?$$

$$V(\pi_{\text{Expectimax}}^{\text{mini}}, \pi_{\text{min}}) = -2$$

→ Recurrence of Expectiminimax



$$V_{\text{Expecti}}^{\text{minimax}}(s) = \begin{cases} \text{utility}(s) & \text{isEnd}(s) \\ \max_{a \in \text{Action}(s)} V_{\text{Expecti}}^{\text{minimax}}(\text{succ}(s, a)) & \text{Player}(s) = \text{Agent} \\ \min_{a \in \text{Action}(s)} V_{\text{Expecti}}^{\text{minimax}}(\text{succ}(s, a)) & \text{Player}(s) = \text{Opp} \end{cases}$$

main problem is computation in real life world game so you can't make full tree. b/c game goes too long & branching factor is very high.

→ How to compute value of Game?
- do exhaustive search?

→ Computation:
example: chess Game
branching b , depth- d ($2d$)
↳ 1 ply when both players have done their play.

$O(b)$ = space Complexity
 $O(b^2d)$ = time Complexity

in chess
 $b \approx 35$, $d \approx 50 \rightarrow 50 \times 2 = 100$ levels
& at each level it has 35 branches

For Go Games

$b = 361$, $d \approx 100 \rightarrow$ b/c 2d
 it's not possible for its

tree

(*) we want to speed up Minimax
 (i) Evaluation function $\rightarrow$ uses domain knowledge & compute approximate answer.

ply = no of levels or what will be move of both of players.

(ii) Alpha beta pruning $\rightarrow$ general purpose, compute exact answer.

(i) we use Depth limited tree for evaluation function.

$d \rightarrow$ limit & if $d=0 \rightarrow$ root
 $\rightarrow V_{\minimax}(s, d)$ utility(s) ii End
 $Eval(s)$ $d=0$
 here $d \neq 0$
 $\max_{a \in A(s)} V_{\minimax}(succ(s, a), d) \text{ Player } (P) = \max$
 $\min_{a \in A(s)} V_{\minimax}(succ(s, a), d-1) \text{ Player } (P) = \min$

if your limit = 3, means only 3 levels.

$Eval(s)$ returns the situation of opp's your position.

$Eval(s) \rightarrow$ Estimate of $V_{\minimax}(s)$

$\rightarrow Eval(s)$ in chess game

$Eval(s) = \text{material} + \text{mobility} + \text{King Safety}$
 (pieces) (freely you or opp can move)

central control.

material = $100(K - K')$
 if your king there and opp not
 100, if your gone (-1)

material = $100(K - K') + 9(Q - Q') + 5(R - R') + 3(B - B' + N - N') + 1(P - P')$

Mobility = $0-1$ (no of legal move - no of legal moves)
 (my moves) (opp moves)

so $eval(s)$ is trying to estimate $V_{\minimax}$.

(ii) Strategy Alpha beta pruning:
 choose A or B with max value

A B
 $[3, 5]$ $[5, 100]$

we don't know what opp will return
 so for this we will go to B b/c
 $B_{\min} > A_{\max}$ so from this,
 we exclude those branches which
 have no effect in your tree.

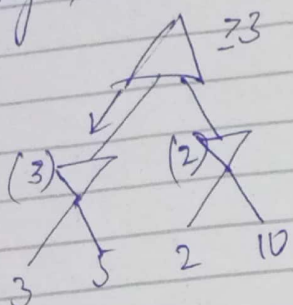
so you don't look at A.

~~b/c~~ $3 \sim 5 \sim 100$

↳ non overlapping ranges.

proof is easy b/c if $R[4,10]$ we have to go to A (overlap)

→ Pruning Game Tree:



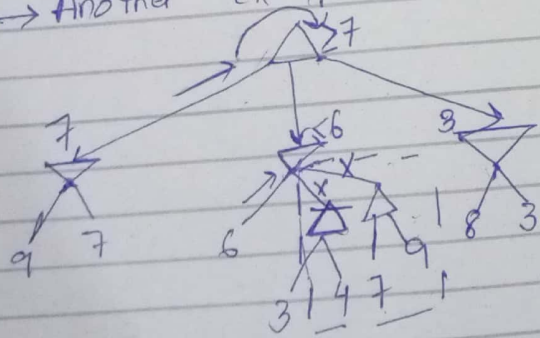
if I explore only left I know that value will be ≥ 3

for right branch value will be ≤ 2

so don't go to (10)

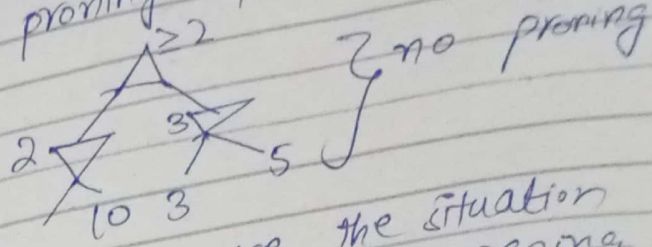
so 10 will be pruned.

→ Another example:



b/c Δ rank change ≥ 7 so prune it

problem is ordering of nodes
→ α - β pruning depends on ordering



so we have to see the situation in which pruning is happening then apply it.

→ Complexity Analysis:

- worst ordering $O(2^{2d})$ time
- Best ordering $O(2^{d \times 0.5})$ time
- Random Ordering $O(2^{d \times 0.75})$ time

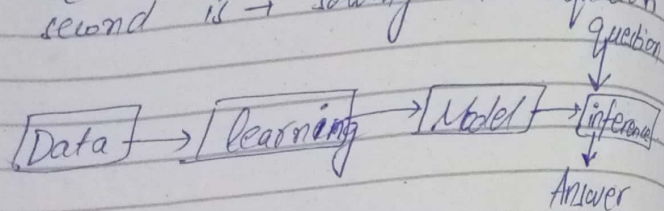
Games Topic Ended

Thursday

Logic:
Knowledge representation of real world
↳ we do reasoning on it.

→ Question:
if $x_1 + x_2 = 10$ and $x_1 - x_2 = 4$, what
is x_1 ? or → search

is XI?
one mechanism is $\rightarrow$ search
second is $\rightarrow$ solving their equation



$Wf = 2x_1 + 5x_3$ $\rightarrow$ this is model
but it is not human readable
when you give x_1 & x_3 values then
it gives some value.

→ for human interable, when you see model, you get to know what is meant by that model that are:

→ logic based Model

- ↳ in this we use propositional logic, first order logic.
- "human is informal logic".

- Applications: → theorem proving, verification, reasoning
- we think in terms of logical

formulæ & inference rules

Problems?

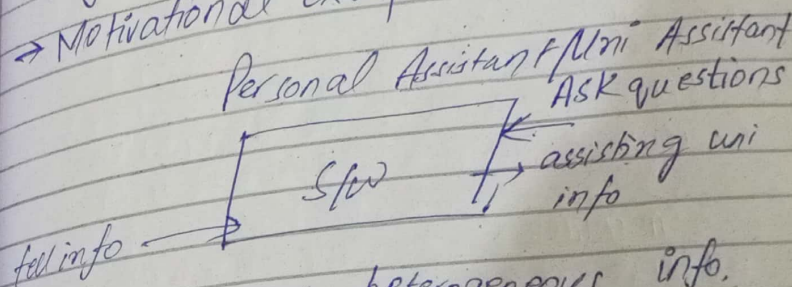
Problems?
 ① Rules are deterministic (no uncertainty if solved) but real world is uncertain - and that is solved by heuristics.
 ② did not allow fine grained solis machine learning

② Rule based - did not allow fine tuning using data. like sofie Machine learning is a compact

strength:

strength;
it provides expressiveness in a compact way.

→ Motivational example:



→ need to digest heterogeneous info.
to give all ans)

→ need to digest (not possible to give all ans)

→ Software should reason deeply with that info.

why not using natural language?

→ b/c it is ambiguous

Example:

-A dime is better than a nickel

[illegible]

- therefore Dime is better than penny
 Example (of confusion):
 - A penny is better than nothing
 - Nothing is better than World peace
 → therefore penny is better than World peace (x wrong)
 Natural language can't be used in programming.

→ Logic Based Model:

Goals:

- 1) Represent knowledge about the world
- 2) Reason with that knowledge.

→ Ingredients of logic:

- Syntax → set of valid formulae.
 example: Rain $\wedge$ Wet ($\wedge$ = and)

- Semantics → for each formulae specify set of models (assignment configuration of the world)

		0	1	Wet
Rain	0	0	0	
	1	0	1	

{ when both 1, then it will comply

→ inference rules, deriving new formulae from giving formulae.

ex: Rain $\wedge$ Wet → derive Rain, Wet
 can't derive → Rain, → Wet

Syntax & Semantics:

$2+3, 3+2$
 Two different syntax but same semantics.

same syntax but diff semantics:
 python version $(2 \cdot 7) \rightarrow 3/2$ (int division)
 python 3 $\rightarrow \frac{3}{2} \neq 1.5$ (float division)

→ Propositional logic:

- syntax:
 - symbols: $(A, B, C, \dots)$ → atomic formulae
 - logical connections: $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$

- we build formulae recursively -
 f.g is a formulae than
 → f also formulae

- f $\wedge$ g " "
- f $\vee$ g " "
- f $\leftrightarrow$ g " "
- f $\rightarrow$ g " "

Here should be syntactically incorrect for $f \neg g$ (x) wrong.

formulae is syntax (don't convey any meaning).

→ Models:

models in prop logic is an assign of truth values to propositional symbols

Ex:

→ 3 prop symbols (A, B, C)
 $2^3 = 8$ possible models.

if $A=0, B=0, C=0$

$$A \wedge B = 0$$

$$A \vee \neg B = 1$$

what formula will return universe
 interpretation function.

→ Interpretation fun:

let f be a formula

let w be a model (will take values)

$I(f, w)$ return:

- true (1) [if w satisfies f]

- false (0) [if w does not satisfy f]

Ex:

$$f = (\neg A \wedge B) \leftrightarrow C$$

$$\text{Model } w = \{A=1, B=1, C=0\}$$

Interpretation:

$$I((\neg A \wedge B) \leftrightarrow C, w) = 1$$

$$I(\neg A \wedge B, w) = 0$$

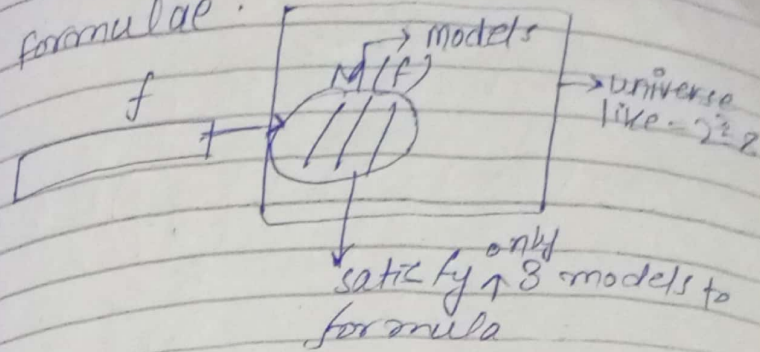
$$I(C, w) = 0$$

$$I(\neg A, w) = 0$$

$$I(B, w) = 1$$

$$I(A, w) = 1$$

no single multiple models satisfy condition
 formulae.



- formula represent set of models.

Ex: $f = \text{Rain} \vee \text{wet}$

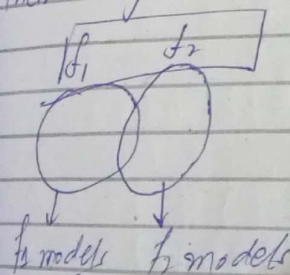
Models

	0	1
Rain 0	0	1
Rain 1	1	1

this formula represent 3 models.

→ Knowledge base (KB):

it is set of formulae representing their conjunction / intersection.

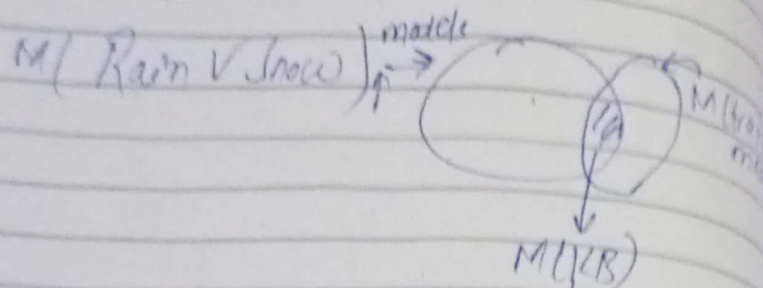


for knowledge base for models
 it represents both

$M(KB)$: → knowledge base has
 many set model satisfy var
 rahel.

$$M(KB) = \bigcup_{f \in KB} M(f)$$

$KB = \{ \text{Rain}, \text{Snow}, \text{Traffic} \}$



jab rain ya snow ho to traffic

Ex:

$M(\text{Rain})$

↑ wet ↓

0	0
1	1

$M(\text{Rain} \rightarrow \text{Wet})$

↑ wet ↓

0	1	1
1	0	1

$M(\text{Rain}, \text{Rain} \rightarrow \text{Wet}) \Rightarrow$

0	0
0	1

> Adding formulas to Knowledge base

Addition

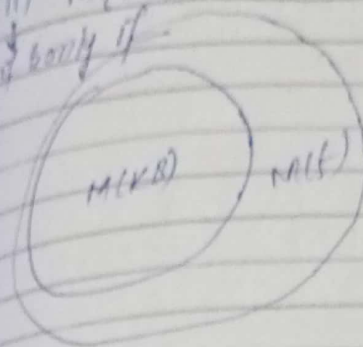
$$KB \rightarrow KB \cup \{f\}$$

Subtract (remove)

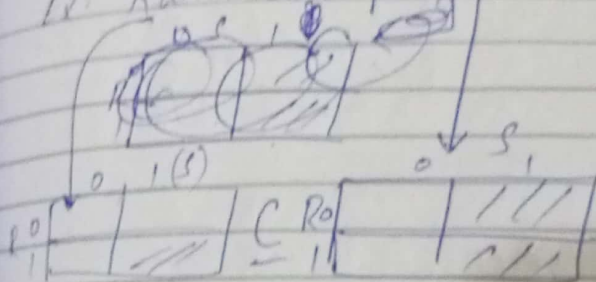
$$M(KB) \rightarrow M(KB) \cap M(f)$$

if you add formula

if KB entails f (KB $\models f$)
 then $M(KB) \subseteq M(f)$
 if body if -

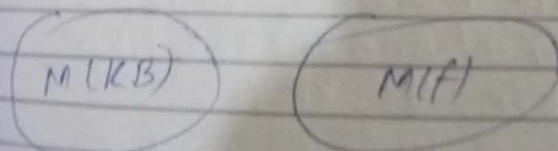


Ex: $\text{Rain} \wedge \text{Snow} \models \text{Snow}$



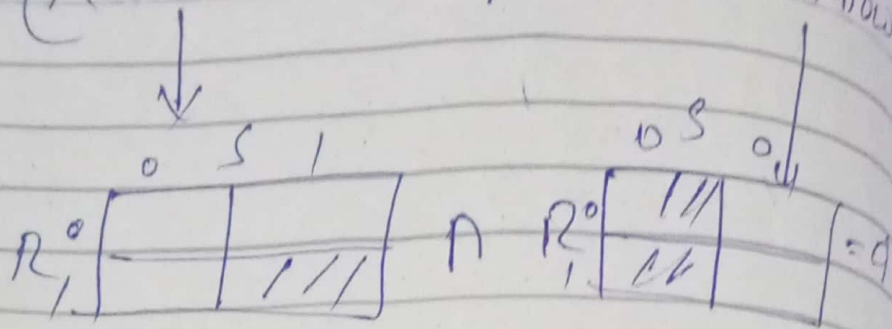
new formulas are subset of KB

contradiction

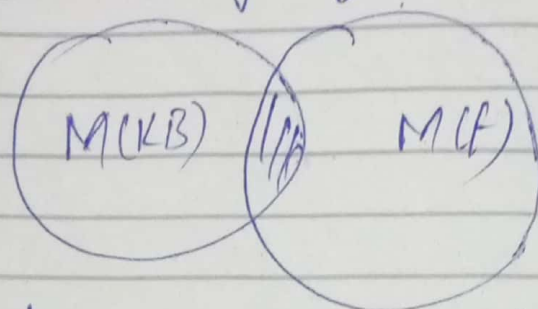


KB contradicts f if
 $M(KB) \cap M(f) = \emptyset$

Ex: (Rain $\wedge$ Snow) Contradict $\rightarrow$ Snow



(ii) Contingency:



$$\emptyset \subset M(KB) \cap M(F) \subset M(KB)$$

neither entailment nor contradict

info. is given through
 $\rightarrow$ Tell information.

Tell(F) $\rightarrow$ [KB] $\rightarrow$?

Possible outcomes. Ex: (it is raining)

$\rightarrow$ I already know that (Entailment)

$\rightarrow$ I don't believe that (Contradiction)

$\rightarrow$ Learned something new (Contingency)
 update the KB.

$\rightarrow$ Ask operation:

is it raining?

$\rightarrow$ Yes (entail)

$\rightarrow$ No (contradict)

$\rightarrow$ I don't know (Contingency)